DAV

Damian Chrzanowski

[2025-11-03 Mon 13:03]

Contents

1 Exam bits

- Operate on dataframes
- Use matplotlib for graphing.
- GoSales sample, plus more uploaded today
- That's it

2 Pandas

2.1 Handy tabulate function

```
def printDF(df):
    print(tabulate(df, headers='keys', tablefmt='pretty', showindex=False))
    print("\n")
```

2.2 Timing the load time

```
start = time.time()
# some work to be done here
end = time.time()
time_taken = end - start
print('Time taken: {0} seconds'.format(time_taken))
```

2.3 Loading data into df and saving to csv

```
df = pd.DataFrame(data) # dict in the key: [array] format
print(df)
df.to_csv("data.csv", index=False) # save to csv
```

2.4 Reading basic

```
# read in the data
data = pd.read_csv("employees.csv")
print("All contents")
print(data)

# print info
print(data.info(verbose=True,memory_usage='deep',show_counts=True))

# nunique - return number of unique values in each column
print(data.nunique())

# unique - return the unique values in each column
print(data.apply(pd.unique))

print("Head")
print(data.head(1))

print("Tail")
print(data.tail(2))

data["Monthly_Salary"] = data["Salary"] / 12 # adding a new column
print(data)

grouped = data.groupby("Department")["Salary"].sum() # grouping data
print(grouped)

# renaming the column of the grouped data
grouped_df = data.groupby("Department", as_index=False)["Salary"].sum() #full dataframe
grouped_df.rename(columns={"Salary": "Total_Salary"}, inplace=True)
print(grouped_df)
```

2.5 Adding Columns

```
# data has price and quantity
df = pdf.DataFrame(data)
df['Total'] = df['Price'] * df['Quantity']

# apply a display format to the floating point numbers
pd.options.display.float_format = '{:.2f}'.format
```

2.6 Calculate sums

```
# data has price and quantity
df = pdf.DataFrame(data)
```

```
# Group the data and sum the quantity sold and the total sales
prodTotals = df.groupby('Product').agg({'Quantity': 'sum', 'Total': 'sum'}).reset_index()
```

2.7 Cleaning data

- `select_dtypes` (type): select all columns with the specific data type, `to_datetime()` : convert to datetime, `to_numeric(columns, errors)` : try to convert to numeric types
- Applying numeric values to certain columns

```
dataF[['Distance', 'Time', 'Calories']] = dataf[['Distance', 'Time', 'Calories']].apply(pd.to_nu
# coerce - converts non numeric to NaN
# ignore - ignore the error and don't change the column
# raise - raise an error, default
```

- Linear interpolating

```
dataF_interpolated = dataF.interpolate()
```

- Date parsing

```
# first extract the columns of interest
date_cols = df[['startdate', 'timestamp']].copy()
print(date_cols.info())
data_cols['startdate'] = pd.to_datetime(date_cols['startdate']).dt.strftime('%d-%m-%y')
data_cols['timestamp'] = pd.to_datetime(date_cols['timestamp']).dt.strftime('%d-%m-%y %H:%M:%S')
```

- Replace wrong values

```
for x in df.index:
    if df.loc[x, 'Duration'] > 120:
        df.loc[x, 'Duration'] = 120
```

- Delete rows where something is wrong or missing

```
for x in df.index:
    if df.loc[x, 'Duration'] > 120:
        df.drop(x, inplace=True)
```

- Drop duplicates

```
# Return true for every row that is a dupe
print(df.duplicated())

# remove dupes
df.drop_duplicates(inplace=True)
```

2.8 Renaming columns

```
# first extract the columns of interest
df.rename(columns={'from': 'To', 'anotherFrom': 'anotherTo'}, inplace=True)
```

2.9 Replacing values in columns

```
# first extract the columns of interest
df['Location'] = df['Location'].str.replace('a', '@')  # replaces all @ with a
```

2.10 Displaying count of missing values

- isnull boolean same-sized object with values that are NA
- notnull same as above but not null
- any(axis=1) return true if any column in a row contains a missing value

```
missing_count = df.isna().sum() # missing values in each column
missing_rows = df[df.isnull().any(axis=1)] # all rows that contain NA values
printDF(missing_rows)
```

2.11 Dropping rows

```
result = df.dropna(how='any')
# drop specific column
result = df.drop(['someColumn'])
```

2.12 Filling missing NaN values using fillna()

```
df['Distance'].fillna(value = df['Distance'].mean(), inplace=True) # fill with mean
```

2.13 Create a new dataframe from a frame

```
activityCalorieSum = dataF.groupby('Category')['Calories'].sum().to_frame()
```

2.14 Summarising

```
query = df.query('Category == "Run"')
totalTime = query['Time'].sum()
totalRuns = len(query.index)
# count instances of things
totalWalks = df['Category'].value_counts()['Walk']
totalWalkDistance = df.loc[df['Category'] == 'Walk', 'Distance'].sum()
totalWalkCalories = df.loc[df['Category'] == 'Walk', 'Calories'].sum()
```

2.15 IAT

```
result['Product'].iat[0] # retrieve value from col Product at row 0
```

2.16 Queries (like SQL where)

- Columns in data: Name (str), Age (float64), Department (str), Salary (int), PerformanceScore (float64), StartDate (date)

```
df = pd.DataFrame(data)
# all employees in a certain dept
it_employees = df.query("Department == 'IT'")
print(it_employees)

# using and / or
it_or_hr = df.query("Department == 'IT' or Department == 'HR'")

# math operators
it_high_salary = df.query("Department == 'IT' and Salary > 50000")

# range operator (here between 45000 and 60000)
mid_salary = df.query("45000 <= Salary <= 60000")

# string matching, starts with 'A'
a_names = df.query("Name.str.startswith('A')", engine='python')

# employees whose name contains the letter 'a'
contains_a = df.query("Name.str.contains('a', case=False)", engine='python')

# the 'IN' operator. Employees in IT or Finance (like SQL IN)
dept_list = ['IT', 'Finance']
selected_depts = df.query("Department in @dept_list")

# Everyone NOT in HR
non_hr = df.query("not Department == 'HR'")

# HR employees with more than 2 years of experience
experienced_hr = df.query("Department == 'HR' and Experience_Years > 2")

# dynamic vars
dept = 'Finance'
min_exp = 5
finance_senior = df.query("Department == @dept and Experience_Years >= @min_exp")

# combinations with parens
# Employees in IT with salary > 50k OR employees in HR with < 3 years experience
combo = df.query("(Department == 'IT' and Salary > 50000) or (Department == 'HR' and Experience_

# sort after query
# Finance department, sorted by experience descending
finance_sorted = (
    df.query("Department == 'Finance'")
```

```
      .sort_values(by="Experience_Years", ascending=False)
)

# math on the query
# Employees whose salary per year of experience > 10,000
# (i.e., Salary / Experience_Years > 10_000)
efficient = df.query("(Salary / Experience_Years) > 10000")

# combining query + adding a new column
# Add a column for Monthly Salary and query those above 4000
df["Monthly_Salary"] = df["Salary"] / 12
monthly = df.query("Monthly_Salary > 4000")
# print only specific columns
print(monthly[["Name", "Department", "Monthly_Salary"]], "\n"

# query + group + aggregate
# Total salary per department, but only for employees with salary > 45000
filtered_group = (
    df.query("Salary > 45000")
      .groupby("Department")["Salary"]
      .sum()
      .reset_index()
      .rename(columns={"Salary": "Total_Salary"})
)
# filter with loc
condition = df['Salary'] > 45000
filtered_df = df.loc[condition]
```

2.17 Analysing data

- sum()
- mean()
- median()
- count()
- std()
- min()
- max()
- pivot(): pivots the data based on the index, columns and values parameters
- cut(): bins the data in equal-sized bins
- nlargest(n=10, columns=['Total Sales']).sort$_{values}$('Total Sales', ascending=False):
- The data has: state, fire$_{year}$, fire$_{month}$, acres$_{burned}$, days$_{burning}$, fire$_{count}$

```
# mean fire
meanFires = fires.groupby('state').mean().round(2)

# better breakdown
meanFires2 = fires.groupby(['state', 'fire_year', 'fire_month']).mean().round(2)

# using aggregation to apply sum, count and mean
result = fires.groupby(['state', 'fire_year', 'fire_month']).agg({'days_burning': ['sum', 'count

# flip the data with pivot()
df.pivot(index='fire_year',columns='state',values='acres_burned').plot() # if we want to plot it
plt.xlabel('Year')
plt.ylabel('Acres burned')
plt.title('Acres burned by year')
plt.show()

# new df from a pivot
new_df = df.pivot(index='fire_year',columns='state',values='acres_burned',aggfunc='sum')

# cutting
binned_data = pd.but(fires_filtered['acres_burned'],bins=4)
```

2.18 Thiago's samples on cleaning

```
# === 1 Load the dirty dataset ===
df = pd.read_csv("employees_dirty_interpolate.csv")
print("=== ORIGINAL DATA (First 10 Rows) ===")
print(df.head(10))

print("\n=== DATAFRAME OVERVIEW ===")
print(f"Number of rows: {df.shape[0]}")
print(f"Number of columns: {df.shape[1]}")

print("\n=== DATA TYPES & MEMORY USAGE ===")
df.info(verbose=True, memory_usage='deep', show_counts=True)

print("\n=== COUNT OF MISSING VALUES PER COLUMN ===")
print(df.isnull().sum())

print("\n=== PERCENTAGE OF MISSING VALUES PER COLUMN ===")
print((df.isnull().mean() * 100).round(2))

print("\n=== UNIQUE VALUE COUNT PER COLUMN ===")
print(df.nunique())

print("\n=== SAMPLE STATISTICS FOR NUMERIC COLUMNS ===")
print(df.describe(include='number'))
```

```
print("\n=== SAMPLE STATISTICS FOR OBJECT COLUMNS ===")
print(df.describe(include='object'))



# === 2 Strip spaces from string columns ===
df["Name"] = df["Name"].str.strip()
df["Department"] = df["Department"].str.strip()
print("\n=== AFTER STRIPPING SPACES ===")
print(df[["Name", "Department"]].head())

# === 3 Identify data types ===
print("\n=== COLUMNS BY DATA TYPE ===")
print("Object columns:", df.select_dtypes(include="object").columns.tolist())
print("Numeric columns:", df.select_dtypes(include="number").columns.tolist())

# === 4 Convert Age and PerformanceScore to numeric ===
df["Age"] = pd.to_numeric(df["Age"], errors="coerce")
df["PerformanceScore"] = pd.to_numeric(df["PerformanceScore"], errors="coerce")
print("\n=== AFTER CONVERTING NUMERIC COLUMNS ===")
print(df[["Name", "Age", "PerformanceScore"]].head())

# === 5 Clean and convert Salary column ===
df["Salary"] = (
    df["Salary"]
    .astype(str)
    .str.replace("[$,]", "", regex=True)
)
df["Salary"] = pd.to_numeric(df["Salary"], errors="coerce")
print("\n=== AFTER CLEANING SALARY ===")
print(df[["Name", "Salary"]])

# === 6 Convert StartDate to datetime (handles all formats safely) ===
s = df["StartDate"].astype(str).str.strip()

df["StartDate"] = (
    pd.to_datetime(s, format="%Y-%m-%d", errors="coerce")    # 2019-03-15
      .fillna(pd.to_datetime(s, format="%Y/%m/%d", errors="coerce"))  # 2020/01/05
      .fillna(pd.to_datetime(s, format="%d/%m/%Y", errors="coerce"))  # 18/04/2018
)

print("\n=== AFTER CONVERTING DATES ===")
print(df[["Name", "StartDate"]])
print("Data type:", df["StartDate"].dtype)

# === 7 Check missing values before filling ===
print("\n=== MISSING VALUES BEFORE CLEANING ===")
```

```
print(df.isnull().sum())

# === 8 Interpolate missing numeric values ===
df[["Age", "PerformanceScore", "Salary"]] = df[["Age", "PerformanceScore", "Salary"]].interpolat
print("\n=== AFTER INTERPOLATION ===")
print(df[["Name", "Age", "PerformanceScore", "Salary"]])

# === 9 Fill remaining missing values ===
df["Department"] = df["Department"].fillna("Unknown")
df["StartDate"] = df["StartDate"].fillna(pd.Timestamp("2020-01-01"))
df["Age"] = df["Age"].fillna(df["Age"].mean())
df["Salary"] = df["Salary"].fillna(df["Salary"].mean())
df["PerformanceScore"] = df["PerformanceScore"].fillna(df["PerformanceScore"].mean())
print("\n=== AFTER FILLING REMAINING NAs ===")
print(df.isnull().sum())

# === 10 Drop duplicates (if any) ===
df.drop_duplicates(inplace=True)

# === 11 Final verification ===
print("\n=== FINAL CLEANED DATA INFO ===")
print(df.info())
print("\n=== CLEANED DATA PREVIEW ===")
print(df)

# === 12 Save cleaned dataset ===
df.to_csv("employees_interpolated_clean.csv", index=False)
print("\n Cleaned file saved as employees_interpolated_clean.csv")
```

2.19 Thiago's samples on analysing

```
# === LOAD DATA ===
sales = pd.read_csv("sales_analysis.csv", parse_dates=["OrderDate"])

print("\n=== HEAD (first 5 rows) ===")
print(sales.head())

print("\n=== INFO ===")
sales.info()

print("\n=== BASIC STATS (numeric columns) ===")
print(sales.describe())

# ------------------------------------------------
# GROUPBY + AGGREGATION
# ------------------------------------------------
print("\n=== Total Revenue by Region & Category ===")
by_region_cat = (
```

```
    sales
    .groupby(["Region", "Category"], as_index=False)["Revenue"]
    .sum()
)
print(by_region_cat)

print("\n=== Mean Units and Mean Profit by Category ===")
by_cat = sales.groupby("Category", as_index=False).agg(
    mean_units=("Units", "mean"),
    mean_profit=("Profit", "mean")
)
print(by_cat)

# ------------------------------------------------
# PIVOT TABLE
# ------------------------------------------------
print("\n=== Pivot table: sum Revenue by Region x Category ===")
pvt = pd.pivot_table(
    sales,
    index="Region",
    columns="Category",
    values="Revenue",
    aggfunc="sum",
    fill_value=0
)
print(pvt)

print("\n=== Pivot table: avg Profit by Channel x Category ===")
pvt2 = pd.pivot_table(
    sales,
    index="Channel",
    columns="Category",
    values="Profit",
    aggfunc="mean",
    fill_value=0
)
print(pvt2)

# ------------------------------------------------
# BINNING WITH cut() AND qcut()
# ------------------------------------------------
print("\n=== Binning UnitPrice into 4 equal-width bins (cut) ===")
price_bins = pd.cut(sales["UnitPrice"], bins=4)
print(price_bins.value_counts(dropna=False))

print("\n=== Binning Revenue into 4 quantiles (qcut) ===")
rev_bins = pd.qcut(sales["Revenue"], q=4, duplicates="drop")
print(rev_bins.value_counts(dropna=False))
```

```
# ------------------------------------------------------
# TOP N (nLARGEST)
# ------------------------------------------------------
print("\n=== Top 5 orders by Profit ===")
top_profit = sales.nlargest(5, "Profit")
print(top_profit[["OrderID", "Region", "Category", "Revenue", "Profit"]])

# ------------------------------------------------------
# PERCENT CHANGE (monthly revenue per region)
# ------------------------------------------------------
print("\n=== Monthly Revenue and Percent Change per Region ===")
monthly = (
    sales
    .set_index("OrderDate")
    .groupby("Region")["Revenue"]
    .resample("MS")  # MS = Month Start
    .sum()
    .reset_index()
)
monthly["PctChange"] = monthly.groupby("Region")["Revenue"].pct_change()
print(monthly.head(12))

# ------------------------------------------------------
# RANKING
# ------------------------------------------------------
print("\n=== Ranking orders by Profit (1 = lowest) ===")
sales["ProfitRank"] = sales["Profit"].rank(method="average", ascending=True)
print(sales[["OrderID", "Profit", "ProfitRank"]].head())

# ------------------------------------------------------
# PLOTS (Matplotlib only, no seaborn)
# ------------------------------------------------------

# 1) Bar chart: total Revenue by Region
print("\n=== Plot: Total Revenue by Region ===")
rev_by_region = sales.groupby("Region")["Revenue"].sum()
plt.figure()
rev_by_region.plot(kind="bar")
plt.title("Total Revenue by Region")
plt.xlabel("Region")
plt.ylabel("Revenue")
plt.tight_layout()
plt.show()

# 2) Line chart: monthly Revenue for one Region (e.g. Leinster)
print("\n=== Plot: Monthly Revenue for Leinster ===")
leinster_monthly = (
```

```
    sales[sales["Region"] == "Leinster"]
    .set_index("OrderDate")["Revenue"]
    .resample("MS")
    .sum()
)
plt.figure()
leinster_monthly.plot(kind="line", marker="o")
plt.title("Monthly Revenue - Leinster")
plt.xlabel("Month")
plt.ylabel("Revenue")
plt.tight_layout()
plt.show()
```

3 Matplotlib

3.1 Prepare data

```
df = pd.read_csv("employees.csv")

# Group by Department and sum the Salary
grouped_df = df.groupby("Department", as_index=False)["Salary"].sum()
grouped_df.rename(columns={"Salary": "Total_Salary"}, inplace=True)
```

3.2 Bar Chart from Employees data

```
# Create a bar plot
plt.bar(grouped_df["Department"], grouped_df["Total_Salary"], color="skyblue", edgecolor="black"

# Add labels and title
plt.title("Total Salary by Department", fontsize=14, fontweight="bold")
plt.xlabel("Department", fontsize=12)
plt.ylabel("Total Salary (€)", fontsize=12)

# Add value labels on top of bars
for i, value in enumerate(grouped_df["Total_Salary"]):
    plt.text(i, value + 1000, str(value), ha='center', va='bottom', fontsize=10)

# Display the plot
plt.tight_layout()
plt.show()
```

3.3 Double Stacked Bar Chart from Employees data

```
# Create a bar plot with stacked bars
plt.bar(prodTotals['Product'], prodTotals['Qty'], label='Quantity Sold', color='blue',bottom=pro
plt.bar(prodTotals['Product'], prodTotals['Total'], label='Total Sales', color='red')
```

3.4 Pie Chart from Employees data

```
plt.pie(
    grouped_df["Total_Salary"],  # values
    labels=grouped_df["Department"],  # labels
    autopct="%1.1f%%",  # show percentages
    startangle=90,  # rotate so first slice starts at top
    colors=["#66b3ff", "#99ff99", "#ffcc99"]
)

plt.title("Total Salary Distribution by Department")
plt.axis("equal")  # make sure pie is a circle
plt.tight_layout()
plt.show()
```

3.5 Line graph

```
plt.figure(figsize=(13, 8))
# first two params are x axis and y axis respectively
plt.plot(montly_data['Month'], monthly_data['Total'], label='Revenue Totals by Month', marker='o
plt.xlabel('Month')
plt.ylabel('Totals')
plt.title('Total by Month')
plt.xticks(rotation=0)
plt.tight_layout() # fits the plot better
plt.savefig("totals.png") # save as png
plt.show()
```